CIS 678 - Machine Learning

– Linear Models: Linear Regression and Perceptron –

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Last Class

- Ensemble learning: why combine models?
- Bagging and random forests
- Boosting and AdaBoost
- Gradient boosting and XGBoost
- Handling class imbalance

Agenda

- Classification vs. Regression
- Linear Regression
  - Linear Model
  - Least Squares
  - MLE and Regularization
- From Regression to Classification
- Perceptron

Classification vs. Regression

Classification

Predict a discrete (categorical) label Y from input X.



X = Email

→ Spam / Not Spam

Y = Class Label

"The movie was fantastic!" ★★★★★

X = Review (Text)

→ Positive / Negative

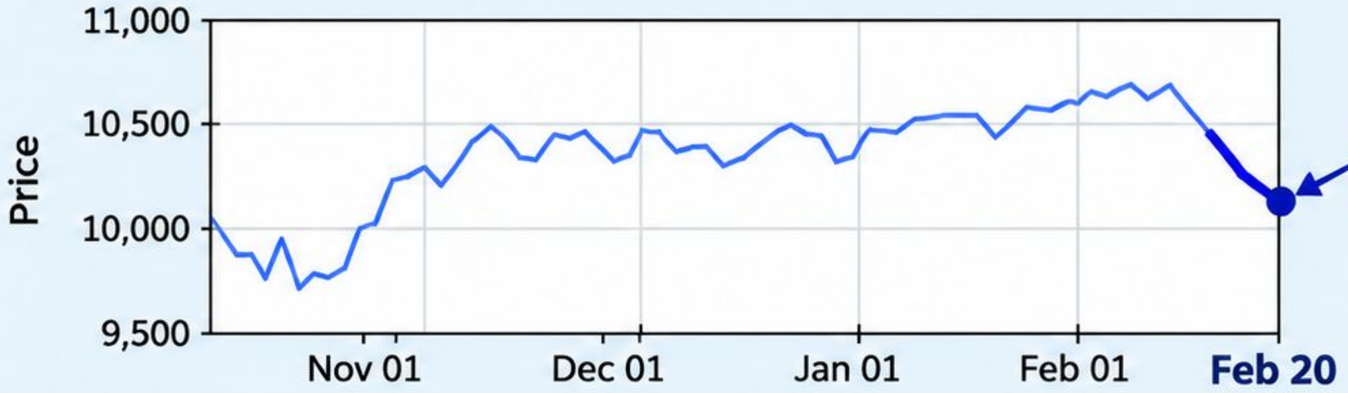
Y = Sentiment

Regression

Predict a continuous (numeric) value Y from input X.

Stock Price Prediction





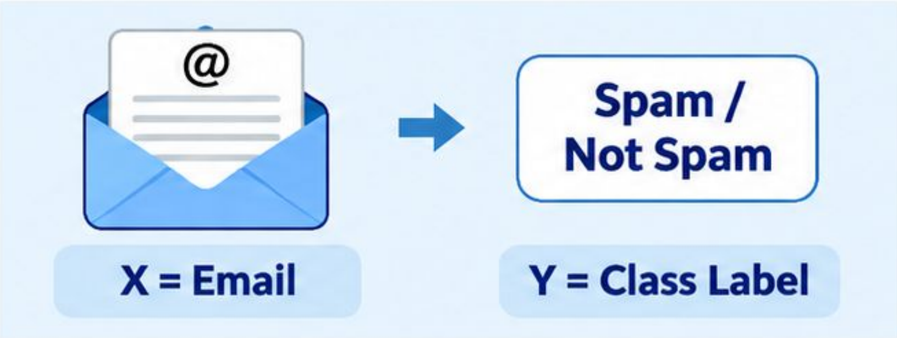
Y = ? (Stock Price)

X = Feb 20

Supervised Learning

Goal: Construct a predictor f: X → Y to minimize a risk (error measure) err(f).

Typical Loss / Error Measures





Classification:
Probability of Error

$$err(f) = P(f(X) \neq Y)$$

Regression:
Mean Squared Error

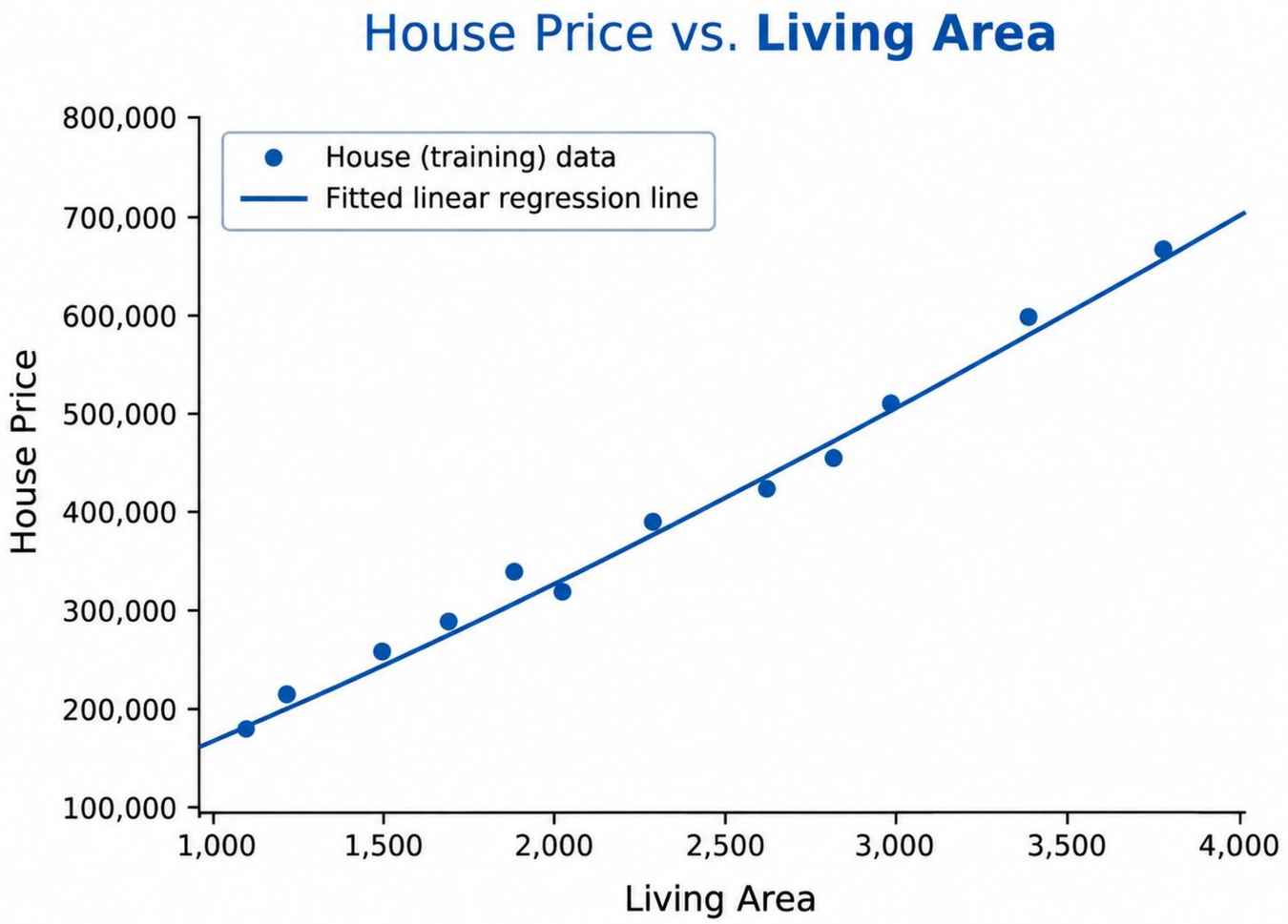
$$err(f) = E[(f(X) - Y)^2]$$

What is Linear Regression?

Linear regression models a continuous target as a linear function of one or more input features.

Examples:

- Predicting house prices based on the living area
- Estimating income based on education, major, and GPA, etc.



Linear Regression Model

How do we determine the best values of w and b from the training data?



For each example, let the input feature vector be

$$x = (x_1, x_2, \ldots, x_d)$$

Linear regression predicts a continuous target using

$$\hat{y} = w^T x + b$$

where

$$w = (w_1, w_2, \ldots, w_d)$$

is the weight vector and b is the bias (intercept).

Equivalently,

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

- x_j: the j-th input feature
- w_j: the weight associated with feature x_j
- b: the bias/intercept
- $\hat{y}$: the predicted value

Each feature contributes $w_j x_j$ to the prediction.

For one input feature, $\hat{y} = wx + b$, which is the equation of a straight line.

Determining the Optimal Weights and Bias

How do we determine the best values of w and b from the training data?

Objective: To learn the "best" weight vector w and bias scalar b from the training data. This allows the linear regression model to make the best possible predictions, where the predicted value $\hat{y}_i = w^T x_i + b$ is as close as possible to the observed value y_i (for $i = 1, \ldots, n$).

Method: Least Squares Regression

- **Goal:** Minimize the sum of squared differences between the predicted and observed values.
- **Loss Function:**

$$L(w, b) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - (w^T x_i + b))^2$$

The best weight vector w and bias scalar b are those that minimize $L(w, b)$, representing the total squared error.

Special Case: Single Input Attribute ($p = 1$)

- **Optimal Weight:**

$$w = \frac{\sum_{i=1}^{n} x_i (y_i - \bar{y})}{\sum_{i=1}^{n} x_i^2 - \frac{1}{n} \left(\sum_{i=1}^{n} x_i \right)^2}$$

- **Optimal Bias:**

$$b = \frac{1}{n} \sum_{i=1}^{n} (y_i - w x_i)$$

where $\bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$ is the average observed output value across all n training samples.

Example of Least Squares Regression

Example. we have four training tuples, each represented by a single attribute x_i and an output variable y_i. We want to find the least squares regression model $y = wx + b$ that best predicts the output y based on the input attribute x.

Index (i)	1	2	3	4
Attribute (x_i)	1	3	5	7
Output (y_i)	4	10	14	16

(a) Training tuples

$$w = \frac{\sum_{i=1}^{n} x_i (y_i - \bar{y})}{\sum_{i=1}^{n} x_i^2 - \frac{1}{n} (\sum_{i=1}^{n} x_i)^2}$$

$$b = \frac{1}{n} \sum_{i=1}^{n} (y_i - w x_i).$$



Example of Least Squares Regression

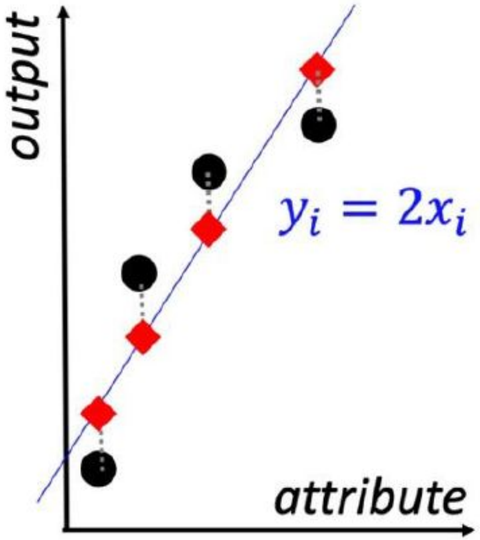
Example. we have four training tuples, each represented by a single attribute x_i and an output variable y_i. We want to find the least squares regression model $y = wx + b$ that best predicts the output y based on the input attribute x.

Index (i)	1	2	3	4
Attribute (x_i)	1	3	5	7
Output (y_i)	4	10	14	16

(a) Training tuples

$$w = \frac{\sum_{i=1}^{n} x_i (y_i - \bar{y})}{\sum_{i=1}^{n} x_i^2 - \frac{1}{n}(\sum_{i=1}^{n} x_i)^2}$$

$$b = \frac{1}{n}\sum_{i=1}^{n}(y_i - wx_i)$$



(b) Least square regression



$$\bar{y} = (y_1 + y_2 + y_3 + y_4)/4 = (4 + 10 + 14 + 16)/4 = 11$$

$$\sum_{i=1}^{4} x_i^2 = 1^2 + 3^2 + 5^2 + 7^2 = 84$$

$$\sum_{i=1}^{4} x_i (y_i - \bar{y}) = 1(4 - 11) + 3(10 - 11) + 5(14 - 11) + 7(16 - 11) = 40$$

$$w = \frac{\sum_{i=1}^{n} x_i (y_i - \bar{y})}{\sum_{i=1}^{n} x_i^2 - \frac{1}{n}(\sum_{i=1}^{n} x_i)^2} = \frac{40}{84 - 64} = 2$$

$$b = \frac{\sum_{i=1}^{4}(y_i - wx_i)}{4} = \frac{(4 - 2\times 1) + (10 - 2\times 3) + (14 - 2\times 5) + (16 - 2\times 7)}{4} = 3$$

Least Squares and Maximum Likelihood Estimation(MLE)

We can also understand Linear Regression from a probabilistic perspective.

Assume the target is generated by a linear model plus Gaussian noise:

$$y_i = w^T x_i + b + \epsilon_i \qquad \text{where } \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Therefore, $y_i \mid x_i \sim \mathcal{N}(w^T x_i + b, \sigma^2)$

Using Maximum Likelihood Estimation (MLE), we choose w and

b to maximize the probability of the observed training data:

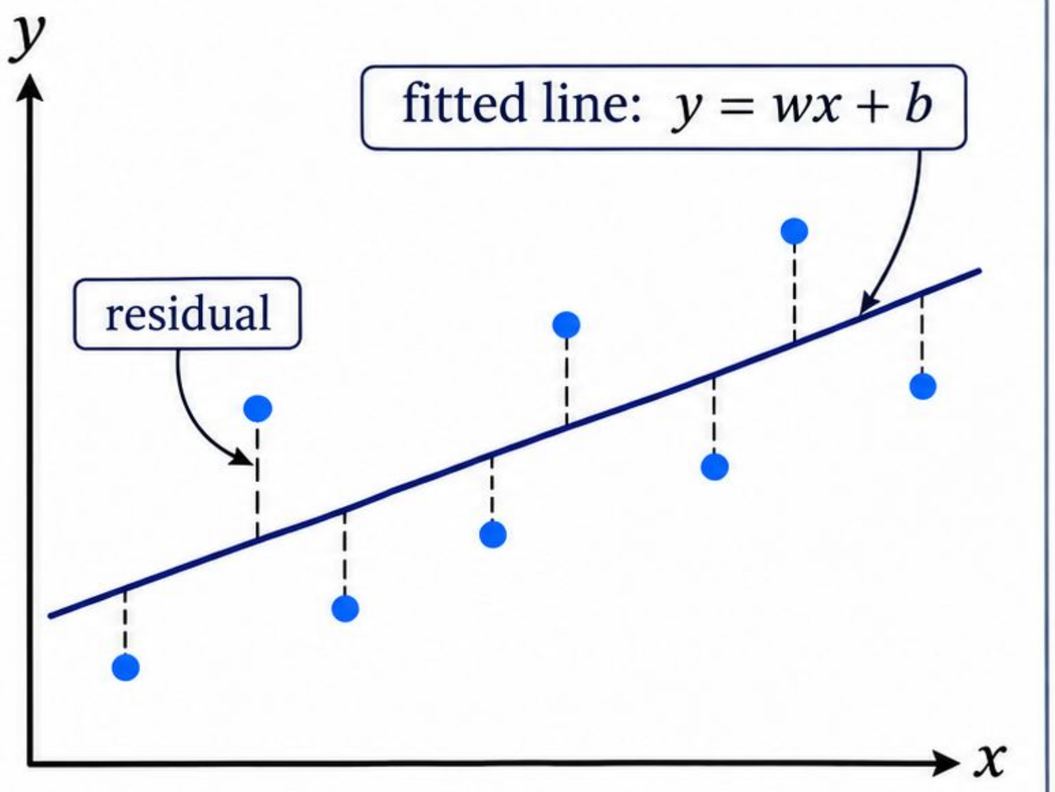
$$(w^*, b^*) = \arg\max_{w,b} \prod_{i=1}^{n} P(y_i \mid x_i, w, b)$$

$$\arg\max_{w,b} L(w, b) = \arg\min_{w,b}$$

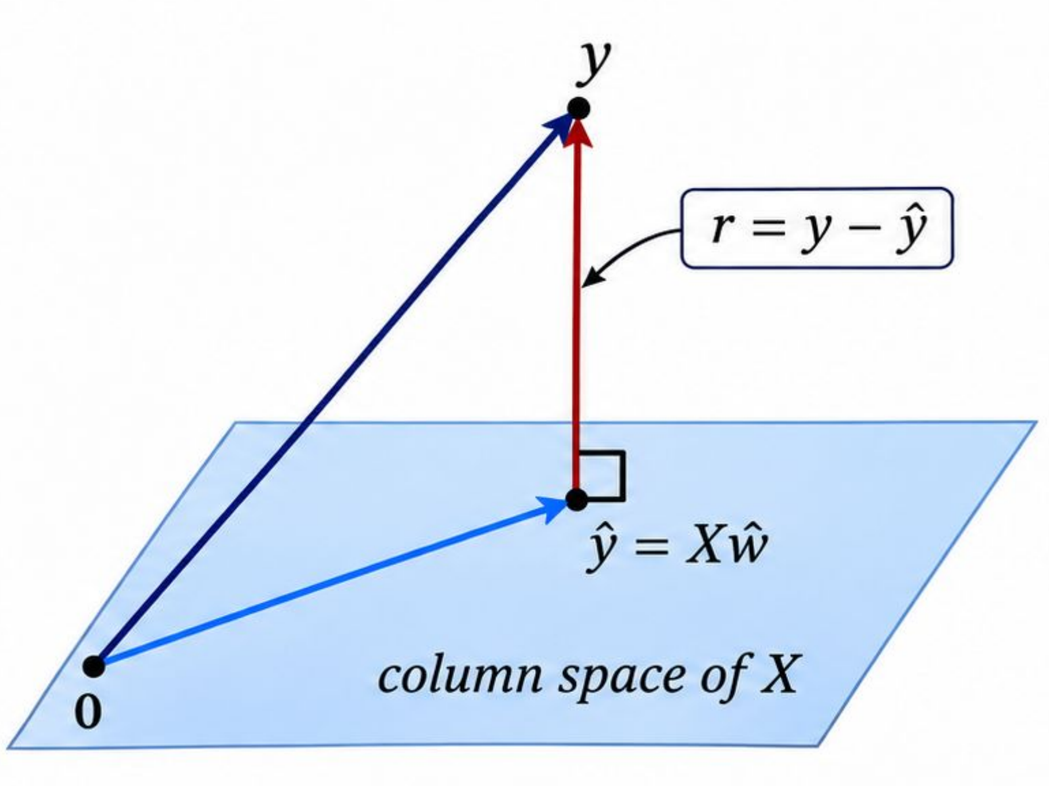
Under the Gaussian noise assumption, maximizing this likelihood is equivalent to minimizing the sum of squared errors:

$$\sum_{i=1}^{n} \left(y_i - (w^T x_i + b)\right)^2$$

Geometric Interpretation



In data space, linear regression fits the line that minimizes squared residuals.



Geometrically, linear regression projects y onto the column space of X.

Regularization: Ridge Regression(L2) MAP with Gaussian prior

Motivation

- Ordinary least squares (OLS) can overfit, especially when features are many or highly correlated.
- Ridge regression adds an L2 penalty to keep the weights small and improve generalization.

Objective (Ridge)

$$L(\mathbf{w}, b) = \sum_{i=1}^{n} \left(y_i - \mathbf{w}^\top \mathbf{x}_i - b\right)^2 + \lambda \|\mathbf{w}\|_2^2$$

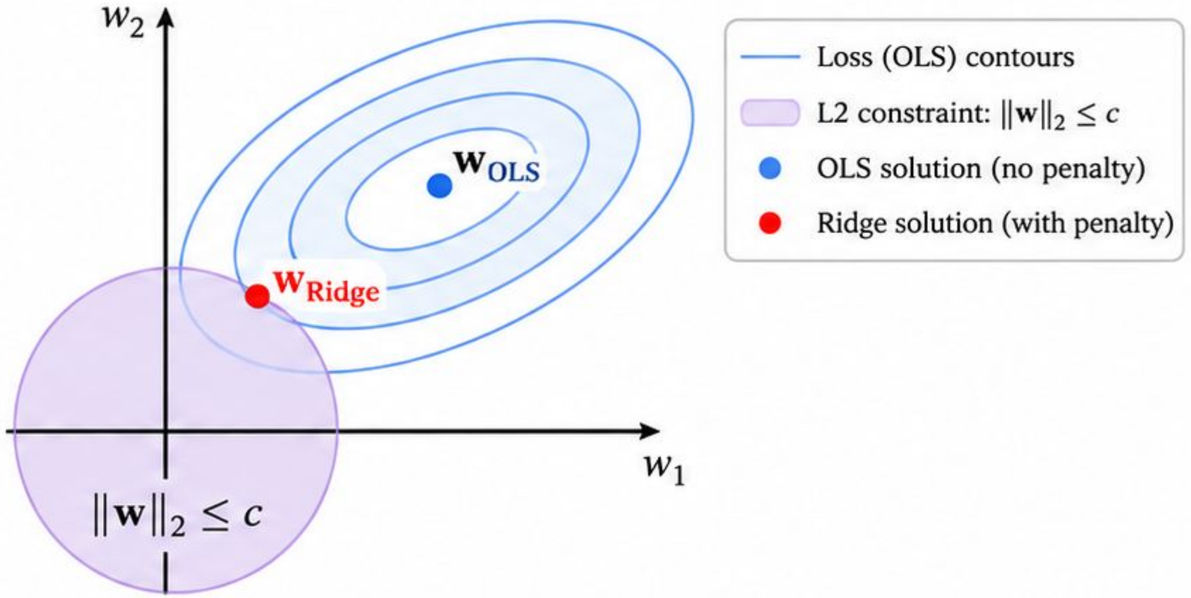
$\lambda \geq 0$ is a regularization parameter that controls the strength of the penalty.

$$\|w\|_2^2 = \sum_{j=1}^{d} w_j^2 \quad \textbf{(L2 norm)}$$

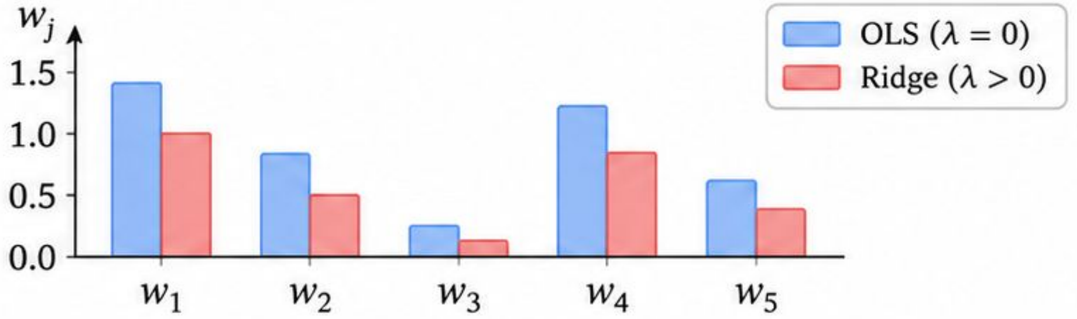
Key Effects of Ridge Regression

- Shrinks weights toward zero (but usually not exactly zero).
- Reduces overfitting and can improve generalization.
- Especially useful when features are many or correlated (multicollinearity).

Geometric View: Minimizing Loss with an L2 Penalty



Effect on the Weights



Regularization: Lasso Regression(L1) MAP with :Laplace prior

Motivation

- Ridge (L2) shrinks weights but rarely sets them exactly to zero.
- Lasso (L1) uses an L1 penalty to encourage sparsity, which can perform feature selection (automatically remove less useful features).

Objective (Lasso)

$$L(\mathbf{w}, b) = \sum_{i=1}^{n} \left(y_i - \mathbf{w}^\top \mathbf{x}_i - b\right)^2 + \lambda \|\mathbf{w}\|_1$$

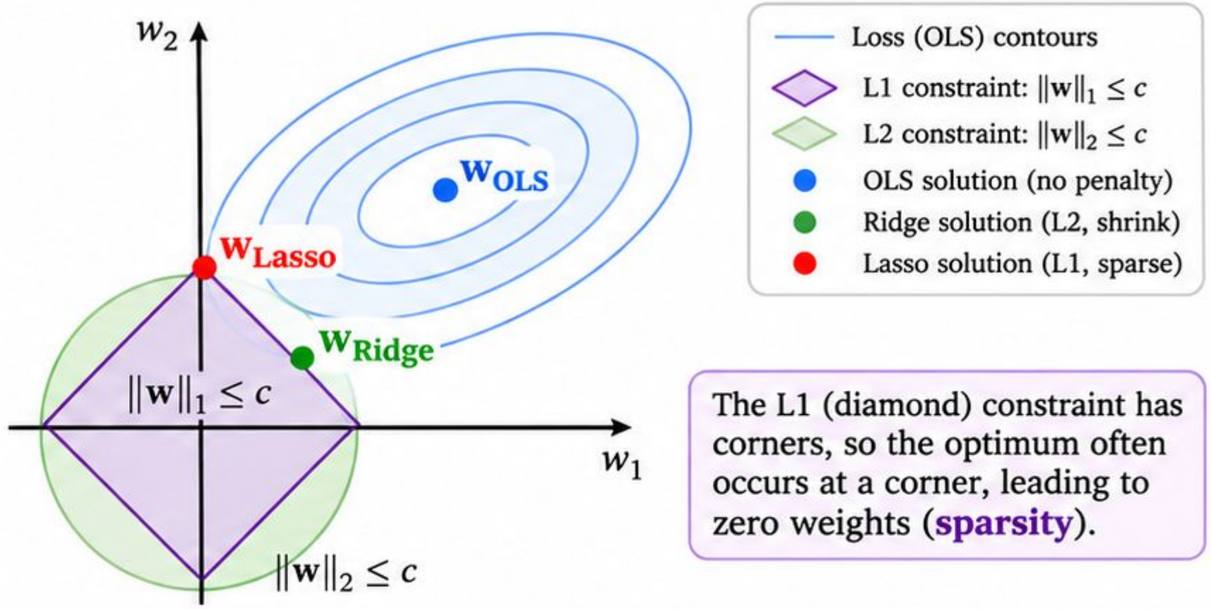
$$\|\mathbf{w}\|_1 = \sum_{j=1}^{d} |w_j| \quad \text{(L1 norm)}$$

$\lambda \geq 0$ controls the strength of the penalty.

Key Effects of Lasso Regression

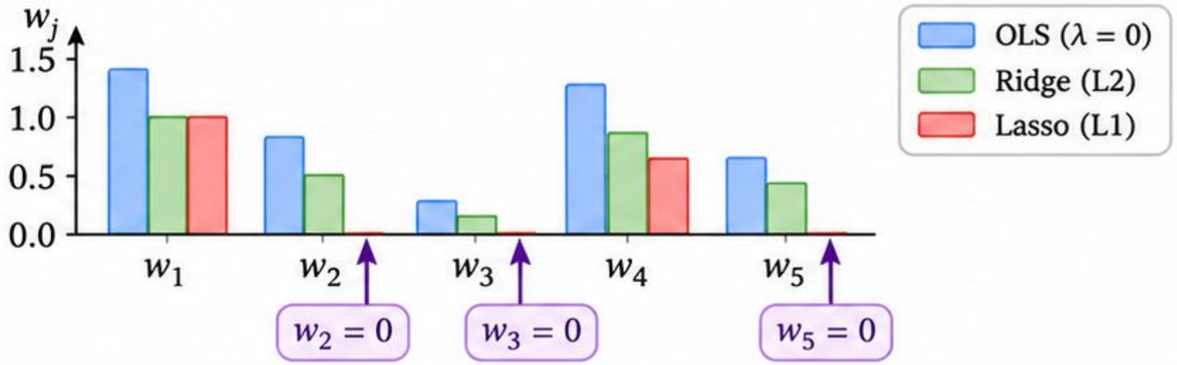
- Encourages sparse solutions (some weights are exactly zero).
- Performs implicit feature selection, which can improve interpretability.
- Often useful when the number of features is large (e.g., high-dimensional or noisy data).

Geometric View: L1 vs. L2 Regularization



The L1 (diamond) constraint has corners, so the optimum often occurs at a corner, leading to zero weights (**sparsity**).

Effect on the Weights



From Regression to Classification

$$z = w^T x + b$$

Regression: use z directly as the predicted numeric value.

Classification: ?



Can a linear score separate two classes?

From Linear Regression to Linear Classification

Motivation

- Linear regression predicts a continuous value:

$$\hat{y} = \mathbf{w}^{\mathrm{T}}\mathbf{x} + b$$

- For binary classification ($y \in \{0, 1\}$), we can use the same linear score for prediction by thresholding.

Linear Classifier

$$s(\mathbf{x}) = \mathbf{w}^{\mathrm{T}}\mathbf{x} + b \quad \text{(linear score)}$$

$$\hat{y} = \begin{cases} 1, & \text{if } s(\mathbf{x}) \geq 0 \\ 0, & \text{if } s(\mathbf{x}) < 0 \end{cases}$$

- The decision boundary is the set of points where $s(\mathbf{x}) = 0$:

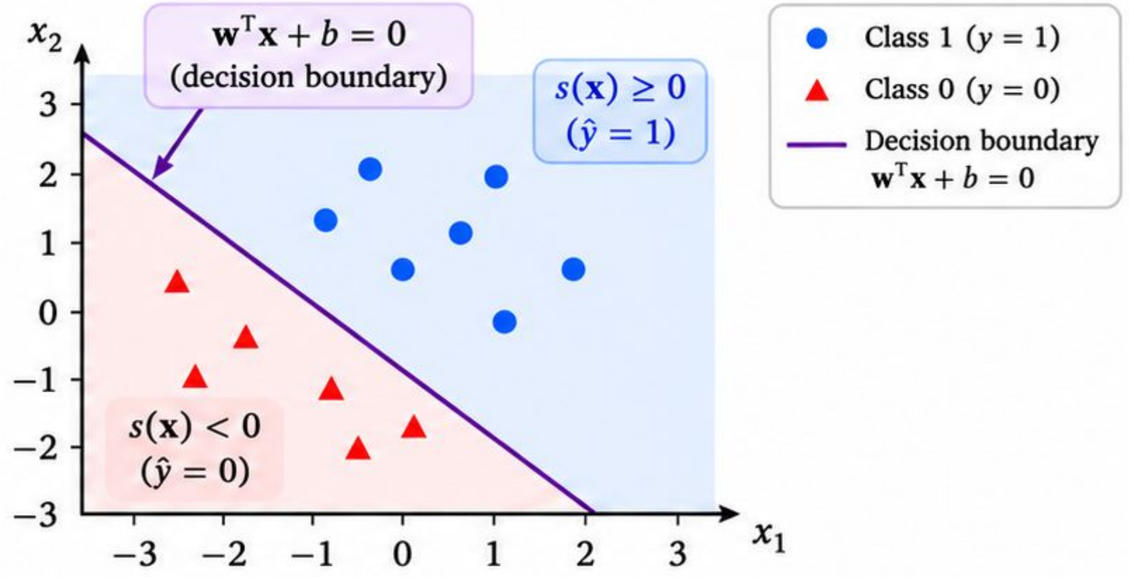
$$\mathbf{w}^{\mathrm{T}}\mathbf{x} + b = 0 \quad \text{(a hyperplane)}$$

- It divides the feature space into two half-spaces: $s(\mathbf{x}) \geq 0$ (predict 1) and $s(\mathbf{x}) < 0$ (predict 0).

Key Takeaways

- Use the linear score $s(\mathbf{x}) = \mathbf{w}^{\mathrm{T}}\mathbf{x} + b$ for classification.
- The decision boundary is a hyperplane.
- This is a simple but effective linear classification model (often called a linear classifier).

Geometric View: A Linear Decision Boundary



Example

Let $\mathbf{w} = [1, -1]^{\mathrm{T}}$, $b = 0$, so $s(\mathbf{x}) = x_1 - x_2$.

$\mathbf{x} = (x_1, x_2)$	$s(\mathbf{x}) = x_1 - x_2$	$\hat{y}$
(2, 0)	2	1
(0, 2)	−2	0
(1, 1)	0	1 (by convention)

Classification Rule

$$\hat{y} = \begin{cases} 1, & s(\mathbf{x}) \geq 0 \\ 0, & s(\mathbf{x}) < 0 \end{cases}$$

Perceptron: From Linear Scores to Classification



How can we find the optimal weight vector w from a set of training tuples?

If the training data are linearly separable, the Perceptron is guaranteed to find a separating hyperplane.

$$y_i \in \{-1, +1\}$$

$$\hat{y}_i = \text{sign}(w^T x_i + b)$$

$$w^T x_i + b > 0 \Rightarrow \hat{y}_i = +1$$

$$w^T x_i + b < 0 \Rightarrow \hat{y}_i = -1.$$

Training Algorithm:

1. **Initialize Weight Vector:**
   - Start with an initial guess for the weight vector w (e.g., set $w = 0$).
2. **Iterative Update Process:**

- Continue updating until convergence or a maximum number of iterations is reached:
  - For each training tuple x_i:
    - **If Prediction is Correct:** No change to w.
    - **If Prediction is Incorrect:**
      - **Positive Tuple** $(+1)$:
        Update $w \leftarrow w + \eta x_i$
      - **Negative Tuple** (-1):
        Update $w \leftarrow w - \eta x_i$
  - Here, η is the learning rate, controlling the size of each update.

Example: 1 for “Y” and -1 for “N”; $\eta = 0.9$

X0	X1	X2	True Label	Predicted Label	W (before update)	W (after update)
1	0	1	Y	N	(0.0, 0.0, 0.0)	
1	1	1	N			
1	0	0	Y			
1	1	0	Y			
1	0	1	Y			
1	1	1	N			
1	0	0	Y			
1	1	0	Y			
1	0	1	Y			
1	1	1	N			
1	0	0	Y			
1	1	0	Y			

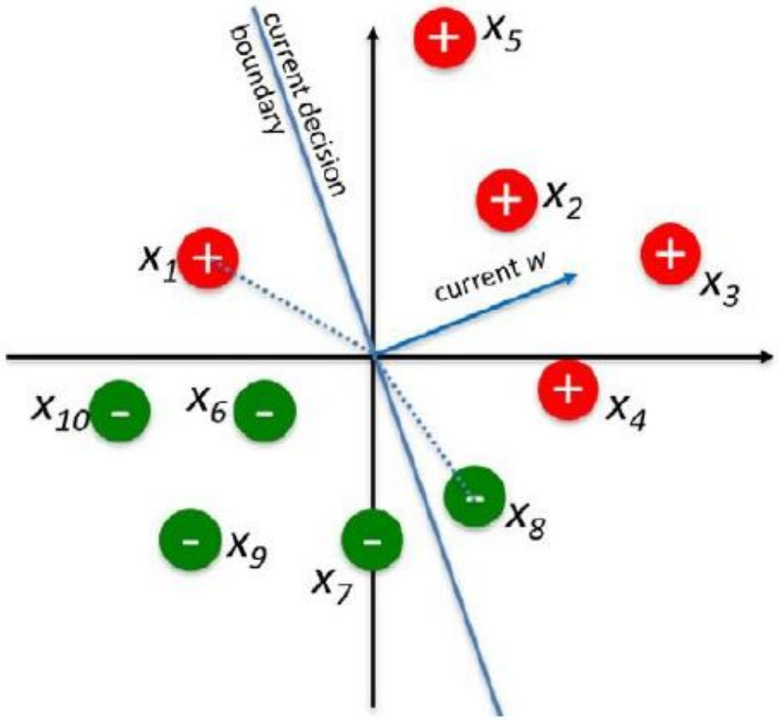


If Prediction is Correct: No change to w.

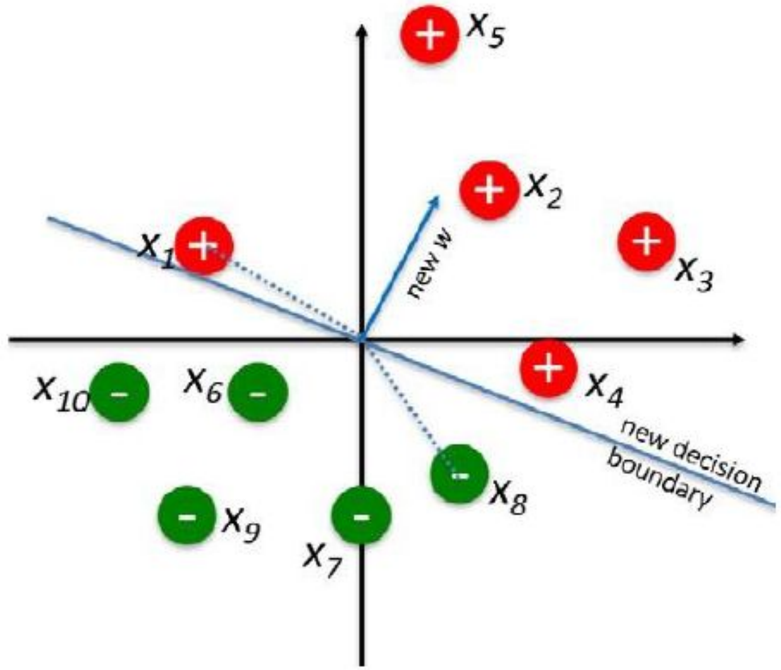
If Prediction is Incorrect:

- **Positive Tuple** $(+1)$:
  Update $w \leftarrow w + \eta x_i$
- **Negative Tuple** (-1):
  Update $w \leftarrow w - \eta x_i$

Perceptron: How an Update Changes the Decision Boundary



(a) current weight vector w



(b) updated weight vector w

Perceptron changes w ⇒ decision boundary changes ⇒ try to correct the mistake

Summary

- Regression predicts **continuous** targets; classification predicts **discrete** labels.
- Linear Regression uses $\hat{y} = w^T x + b$
  - Least Squares learns w,b by minimizing squared error.
  - Under Gaussian noise, Least Squares corresponds to **MLE**.
- Regularization controls model complexity.
  - Ridge: L2 penalty shrinks weights.
  - Lasso: L1 penalty can set some weights to zero.
- Perceptron uses the **sign of a linear score** for binary classification.
  - Perceptron learns the decision boundary by updating w when examples are misclassified.